
Ascend Video Indexing Migration Report

The system under test is one node of eight Ascend 910B3 dies, 64 GB HBM per die, on a Kunpeng-920 aarch64 host running Ubuntu 22.04. The host provides CANN 8.5.2 with the ATC compiler and the pyACL runtime. Two container runtimes were measured: CANN 8.0.RC1 with torch_npu 2.1.0 on Python 3.8, and CANN 9.0.0 with torch_npu 2.10.0 on Python 3.12.

Measurement date is 2026-07-08. All Ascend performance figures were measured on this system. All NVIDIA figures are published reference values and are labelled as such.

Evidence tags used throughout this document:

- [M] measured on this Ascend 910B3 system.
- [R] verified in a named repository, issue, or pull request.
- [P] published reference figure.
- [I] inferred from mechanism, not directly measured.

Executive Summary

The migration is not constrained by the silicon or by the aarch64 architecture. The binding constraints are operator coverage in the eager execution path and version coupling across CANN, torch_npu, Python, and the container runtime. The encoder and convolutional portion of the pipeline reaches production grade performance on the 910B3. The autoregressive portion remains below real time in eager mode.

Production Ready Components

BGE-M3 text embeddings run on torch_npu at 1,577 embeddings per second per die at batch 128, with the AI cores at 97 percent utilisation. Status: Working. [M]

The ArcFace face pipeline runs through the ONNX to OM conversion path at 455 detections per second for the SCRFD detector and 643 recognitions per second for the ArcFace recognizer, executed on a CANN 8.5.2 runtime. Status: Working. [M]

CLIP and SigLIP image embeddings run on torch_npu in fp32 at 741 images per second at batch 32. Status: Working in fp32. [M]

Vector indexing runs from BGE embeddings on the NPU into a local Qdrant collection. Status: Working. [M]

Two large language models run concurrently, Qwen2.5-32B for text and Qwen2.5-VL-32B for vision language, both served through the vLLM Ascend runtime behind a litellm proxy. Status: Working. [M]

The complete Django application runs on the Ascend system. The database schema is migrated, the application programming interface responds, and a sample video is processed through the application code. Status: Working. [M]

Blocked Items

Whisper large-v3 speech recognition produces correct transcripts, but the real time factor reaches only 0.84x in eager mode, which is below real time. Status: Partial. [M]

Open vocabulary YOLO-World detection is not available on the NPU. A fixed class detector is available through the OM path. Status: Partial. [M]

CLIP and SigLIP in fp16 fail on an operator fallback and must run in fp32. Status: Blocked in fp16. [M]

NLLB translation was re-engineered off the original engine and now runs, but at decoder bound throughput. Status: Working, throughput limited. [M]

Items Requiring Huawei Intervention

The fp16 execution path of nn.MultiheadAttention falls back to the host CPU, which blocks fp16 vision transformers. Status: Not resolved. [M][R]

The ascend-docker-runtime does not grant NPU access to newly created containers. Status: Not resolved. [M]

The onnxruntime CANN execution provider does not ship an aarch64 wheel for the Python versions used in the on-box containers. Status: Not resolved. [R]

An OM binary is bound to the CANN version that compiled it and does not load under a different CANN version. Status: Not resolved. [M]

Work Completed

The following migration and re-engineering actions were completed and verified on the Ascend system.

Speech recognition was re-engineered. The original CTranslate2 based engine has no CANN backend, so the speech recognition stage was rebuilt on the Hugging Face transformers Whisper implementation running on torch_npu. [M][R]

Text embeddings were migrated. The BGE-M3 model device assignment was changed from the NVIDIA device to the NPU device, with no other change required. [M]

Image embeddings were migrated. CLIP and SigLIP were moved to the NPU. The fp16 path was found to fail, so the working configuration uses fp32. [M]

The face pipeline was converted to the OM path. The SCRFD detector and the ArcFace recognizer were exported to ONNX, compiled with ATC to OM binaries, and executed with pyACL. [M]

Object detection was partially converted. The YOLO-World model was exported with a frozen class list to ONNX and compiled to an OM binary for fixed class detection. [M]

Translation was re-engineered. NLLB was moved from the CTranslate2 engine to the transformers sequence to sequence implementation on torch_npu. [M]

The Whisper encoder was converted to an OM binary to quantify the encoder cost relative to the decoder. [M]

Two large language models were brought up concurrently on the node, a text model and a vision language model, and were wired into the litellm proxy that the application calls. [M]

The full application was brought up on the Ascend system inside a Python 3.12 environment with a matched torch_npu build. The device references across the application source were changed from the NVIDIA device to the NPU device, service host addresses were pointed at the on-node services, the database was migrated, and a sample video was processed through the application code. [M]

Measured Results

All figures below were measured on a single 910B3 die unless a die count is stated. Warm up iterations were separated from steady state. Latency values are steady state median values.

Table 1. BGE-M3 text embeddings, eager torch_npu, fp16, sequence length 128.

Batch	Latency ms	Embeddings per second	Tokens per second	AI core percent
1	47.8	21	2,677	low
8	49.0	162	20,751	rising
16	48.2	307	39,292	rising
32	49.1	652	83,491	rising
64	52.2	1,198	153,343	90
128	80.4	1,577	201,847	97

Table 2. Whisper large-v3 speech recognition, transformers on torch_npu, fp16. Real time factor is audio seconds divided by processing seconds. A value below 1.0 is slower than real time.

Audio seconds	Batch	Processing seconds	Real time factor
10	8	25.3	0.40
30	8	42.6	0.70
60	8	80.2	0.75
90	8	107.0	0.84
60	4	90.7	0.66
60	16	74.8	0.80

Table 3. CLIP ViT-B/32 image tower, transformers on torch_npu, fp32.

Batch	Latency ms	Images per second
1	31.4	30
8	29.0	236
16	29.9	513
32	40.5	741

Table 4. ArcFace face pipeline, ONNX to OM through ATC, pyACL runtime, batch 1.

Model	Role	Input	Latency ms	Inferences per second
SCRFD det_10g	Detection	1x3x640x640	1.32	455
ArcFace w600k_r50	Recognition	1x3x112x112	0.88	643
Full pipeline on CPU	Baseline	Variable	2857	0.35

Table 5. End to end indexing. Real transcript segments were embedded on the NPU and written to a Qdrant collection.

Metric	Value
--------	-------

Segments indexed	500
Index time seconds	9.7
Segments per second	51.7
Vector dimension	1024

NVIDIA Reference Comparison

A literature search returned very few exact model figures for the NVIDIA H100 or H200 on these specific models. The values below use the nearest credible published reference, with the hardware and confidence stated for each row. The H200 is equivalent to the H100 in compute with faster and larger HBM3e memory, so H200 throughput is expected to equal or exceed H100 for these memory bound workloads.

Two of the comparisons are not directly comparable and are marked in the table notes. The Ascend values are single die, eager or OM, without graph mode and without multi die sharding, so they are conservative.

Table 6. Ascend measured values against published NVIDIA reference values.

Component	Ascend 910B3 measured	NVIDIA reference	Reference hardware	Confidence	Comparable
BGE-M3 embeddings	1,577 per second per die, 201k tokens per second, batch 128	About 60,000 tokens per second	A100 80GB, TEI fp16 batch 512	Low	Approximate
Whisper large-v3	0.84x real time, single stream	1,800x to 2,400x real time batched; 8x to 40x single stream	H100, TensorRT-LLM batched	Medium	Not comparable, see note 1
CLIP ViT-B/32	741 images per second, batch 32, fp32	About 290 images per second with server overhead	TITAN RTX, open_clip	Low	Approximate
SCRFD det_10g	455 per second, batch 1, OM	About 204 to 500 per second	RTX 2080 Ti and 2080 Super	Medium	Approximate
ArcFace w600k_r50	643 per second, batch 1, OM	526 per second at batch 1, 1,928 at batch 64	RTX 2080 Super, TensorRT fp16	Medium	Approximate

Table Notes.

Note 1. The Whisper H100 figure is aggregate throughput produced by heavy in-flight batching under TensorRT-LLM. The Ascend figure is single stream eager. The two are not directly comparable. The difference reflects the serving stack, graph compilation and fused kernels, and not the raw silicon.

Note 2. No exact model H100 or H200 published figures exist for CLIP ViT-B/32, SCRFD, ArcFace r50, or BGE-M3, so no specific Ascend to H200 ratio is claimed.

Sources for the reference figures: Baseten and InferenceBench for Whisper on H100; a text

embeddings inference benchmark for BGE-M3 on A100; the SCRFD paper at arXiv 2105.04714 and a TensorRT face pipeline project for SCRFD and ArcFace; open_clip model profile data for CLIP.

Root Cause Analysis

The performance limits observed during the migration trace to a small number of engineering causes.

Eager dispatch overhead is the dominant limit for autoregressive and small batch work. Every torch_npu forward pass carries a fixed host side dispatch cost of about 30 to 48 milliseconds on this stack. For batched encoders this cost is amortised and the AI cores reach 97 percent. For autoregressive decode, every generated token is a separate dispatch, so the cost cannot be amortised and Whisper is limited to about 0.84x real time. The engineering remedy is graph mode or a fused decoder runtime. Eager execution will remain dispatch bound. [M]

An operator coverage gap blocks fp16 vision transformers. On torch_npu 2.1.0, nn.MultiheadAttention in fp16 raises an error because its input projection falls back to the host CPU, and the host has no fp16 general matrix multiply. Every individual vision transformer operator runs correctly on the NPU. Only the fused attention path fails. The working remedies are fp32 execution, a newer CANN and torch_npu combination, or an attention block built on scaled dot product attention. [M][R]

The default ONNX runtime path falls back to the CPU. The stock onnxruntime exposes only the CPU execution provider, so the face pipeline ran at 0.35 images per second. The maintained CANN execution provider does not ship a wheel for the container Python version. The OM path avoids onnxruntime entirely and is the correct remedy, reaching 455 and 643 inferences per second. [M][R]

A framework abstraction gap blocks NPU object detection. The ultralytics package does not expose an NPU device, so YOLO-World runs on the CPU. The remedy is ONNX to OM conversion, which requires freezing the class list. [M][R]

Conversion and runtime coupling limits portability. An OM binary compiled under CANN 8.5.2 does not load under CANN 8.0.RC1. The remedy is to build and run each OM binary under one CANN version and to treat OM inference as its own runtime tier. [M]

Open Issues

Table 7. Open issues ranked by severity.

Number	Severity	Issue	Impact
1	Blocker	ascend-docker-runtime does not grant NPU access to new containers	Cannot create a clean container for the target stack; existing containers are reused
2	Blocker	The original speech and translation engine has no CANN backend	Speech recognition and translation must be re-engineered
3	Major	nn.MultiheadAttention fp16 falls back to the CPU on torch_npu 2.1.0	fp16 CLIP and SigLIP are blocked; fp32 is the workaround
4	Major	No onnxruntime CANN wheel for the container Python	ONNX models run on the CPU unless converted to OM
5	Major	Whisper eager real time	No real time speech

		factor is 0.84x	recognition without graph mode
6	Major	ultralytics has no NPU device and open vocabulary export is hard	Object detection is limited to fixed class OM
7	Moderate	OM binaries are bound to the compiling CANN version	Split runtime deployment and version discipline required
8	Moderate	Python 3.8 in the matched torch_npu container blocks the web framework	Web tier must run under Python 3.12
9	Minor	Some source media files are truncated and crash the image loaders	A data validation guard is required
10	Minor	The application uses CUDA hardware video acceleration flags	Frame extraction runs on the CPU on Ascend

Huawei Action Items

The following items require action by Huawei. They are consolidated from across the migration work and ranked by severity. Each item states the issue, the root cause, the required fix, and the business impact.

Item 1. Blocker. NPU injection into new containers. Issue: the ascend-docker-runtime does not grant NPU access to newly created containers, while previously created containers continue to work. Root cause: a degraded runtime state on the host that fails device registration at container creation. Required fix: repair and harden the ascend-docker-runtime, and provide a documented recovery procedure. Business impact: teams cannot provision a clean, version matched container for a target workload, which forces reuse of existing containers and increases operational risk.

Item 2. Major. fp16 attention on vision transformers. Issue: nn.MultiheadAttention in fp16 falls back to the host CPU and blocks fp16 vision transformers. Root cause: the fused attention path is not intercepted for NPU execution in the torch_npu version present, so an input projection general matrix multiply lands on the CPU where fp16 is unsupported. Required fix: make fp16 nn.MultiheadAttention run natively on the NPU across torch_npu versions. Business impact: this single operator stands between the platform and fast vision transformer inference; the workaround doubles memory and halves throughput.

Item 3. Major. ONNX runtime CANN wheels. Issue: the CANN execution provider for onnxruntime is not installable in the on-node container Python. Root cause: aarch64 wheels are published only for newer Python versions than the containers provide. Required fix: publish onnxruntime CANN aarch64 wheels across Python versions, including the versions shipped in official images. Business impact: without this, every ONNX model falls back to the CPU unless manually converted to OM.

Item 4. Moderate. OM version binding. Issue: an OM binary does not load under a CANN version different from the one that compiled it. Root cause: the OM binary is coupled to the compiler application binary interface. Required fix: decouple OM from the compiler CANN version, or publish clear cross version compatibility guarantees. Business impact: deployments are forced into a split runtime with strict version pinning, which increases maintenance cost.

Item 5. Moderate. First party model runtimes. Issue: there is no maintained first party runtime for real time speech recognition or open vocabulary detection on Ascend. Root cause: these paths depend on community components of varying maturity. Required fix: provide first party,

maintained reference runtimes for Whisper and for open vocabulary detection. Business impact: the highest value latency workloads depend on components with uncertain long term support.

Additional Validation Work

The following work was completed after the initial report and is presented separately from the primary results. It extends the OM path, measures multi die scaling, and confirms the translation re-engineering.

NLLB translation re-engineering was confirmed working. The model was moved to the transformers implementation on torch_npu and produced correct output for English to Arabic at 9.0 sentences per second, English to Malayalam at 6.8 sentences per second, and English to French at 9.9 sentences per second. The stage is decoder bound, consistent with the Whisper finding. [M]

The Whisper encoder was converted to an OM binary and measured at 8.25 milliseconds per 30 second window, which corresponds to roughly 2,940x real time for the encoder alone. This confirms that the sub real time result for Whisper is caused entirely by the autoregressive decoder and that converting the encoder to OM does not remedy it. The remedy is a fused decoder runtime. [M]

A fixed class YOLOv8 detector was converted to an OM binary and measured at 3.6 milliseconds, which is 200 detections per second, compared with about 1 to 2 per second on the CPU path. Freezing the class list removes the open vocabulary capability. [M]

Multi die scaling was measured for the embedding path. BGE-M3 was run concurrently on six free dies and reached an aggregate 5,799 embeddings per second, which is 4.84 times the single die value, or about 81 percent linear scaling. Per die throughput drops about 20 percent under concurrency due to shared memory controllers and host tokenisation. Extrapolated to eight dedicated dies, the node level rate is roughly 7,700 to 9,600 embeddings per second. [M]

Recommendations

Immediate actions. Deploy BGE-M3 on torch_npu across dies, and deploy the ArcFace face pipeline through the OM path on a CANN 8.5.2 runtime. Run CLIP and SigLIP in fp32. Add a data validation guard for truncated media. Document the ascend-docker-runtime recovery procedure with Huawei.

High confidence fixes. Move speech recognition to a graph based or fused runtime, since eager execution will not reach real time. Replace the original translation engine with the transformers implementation on torch_npu, or route translation through the on-node language model. Stand up a CANN 9.0 and Python 3.12 container for the web tier and the torch models once the runtime is repaired.

Experimental items. Validate fp16 CLIP and SigLIP on CANN 9.0 with torch_npu 2.10, which is expected to remove the CPU fallback. Evaluate dense only BGE-M3 on the vLLM Ascend runtime for higher queries per second when the sparse and multi vector outputs are not required. Convert a fixed class YOLOv8 detector to OM and compare it against the open vocabulary requirement.

Huawei dependent items. Native fp16 nn.MultiheadAttention on the NPU. A repaired ascend-docker-runtime. onnxruntime CANN aarch64 wheels across Python versions. OM decoupled from the compiler CANN version.

Deprioritised items. Installing the CANN execution provider inside the Python 3.8 container, since no wheel exists; use OM instead. Porting the original translation engine to CANN, since it has no backend. Using OM for autoregressive decoders, which is the wrong tool. Reviving the stale or wrong target community ports.

Appendix

Table 8. Migration matrix.

Stage	Original engine	Ascend route	Status
Text embeddings	Torch on NVIDIA	Eager torch_npu	Working
Face detection and recognition	ONNX on NVIDIA runtime	ONNX to OM with pyACL	Working
Image embeddings			
Vector index	Torch on NVIDIA	Eager torch_npu fp32	Working
Language model	Qdrant	Local Qdrant	Working
Speech recognition	NVIDIA vLLM	Ascend vLLM, two models	Working
	WhisperX on CTranslate2	transformers on torch_npu	Partial, below real time
		transformers on torch_npu	
Translation	NLLB on CTranslate2	Fixed class ONNX to OM	Working, throughput limited
Object detection	ultralytics YOLO-World	Host CPU	Partial, fixed class only
Video frame acceleration	NVIDIA hardware codec		Working, not accelerated

Table 9. Per component status.

Component	Status	Route	Measured value
BGE-M3	Working	Eager torch_npu	1,577 per die, 5,799 across six dies
ArcFace face pipeline	Working	ONNX to OM with pyACL	455 and 643 per second
CLIP and SigLIP	Working in fp32	Eager torch_npu fp32	741 images per second
Qdrant index	Working	Local Qdrant	51.7 segments per second
			200 per second
YOLOv8 fixed class	Working	ONNX to OM	7 to 10 sentences per second
NLLB translation	Working	transformers torch_npu	0.84x real time
			Not measured
Whisper	Partial	transformers torch_npu	
YOLO-World open vocabulary	Blocked	Requires text path export	
Active speaker detection	Blocked	Code ports, weights missing	Not measured